

Contents

1	Quick Start	3
2	Overview	7
2.1	Installation and Usage	7
2.1.1	Installation	7
2.1.2	Dependencies	7
2.1.3	Usage	8
2.2	Design Goals and Purpose	8
2.3	Overview: Modules	9
2.3.1	User-Level Modules That Use Containers	9
2.3.2	User-Level Modules That Do Not Use Containers	9
2.3.3	Back End Modules	10
3	CoCoT_EX's Design Principles	11
3.1	Containers and Data Types	11
3.1.1	Containers	11
3.1.2	Data Types	11
3.1.3	Inheritance	12
3.2	Components	12
3.2.1	General Concept	12
3.2.2	How to Use Components	13
3.3	Properties	15
3.3.1	General Concept	15
3.3.2	How to Use Properties	15
3.4	Attributes and Style Classes	18
3.4.1	Attributes	18
3.4.2	Style Classes	18
3.5	Optional Macro and Environment Prefix	18
3.6	Accessibility Features	19
 Part I CoCoT_EX's Container-Based Modules		
4	coco-headings.sty	21
5	coco-floats.sty	22
6	coco-title.sty	23
7	coco-lists.sty	24

Part II Additional Modules

8	coco-scripts.sty	26
9	coco-endnotes.sty	27
10	coco-accessibility.sty	28

Part III CoCoT_EX's Backend

11	coco-kernel.sty	30
12	coco-common.sty	32
12.1	Using Properties	32
12.1.1	Setting and Using Property	32
12.1.2	Default Property Sets	32
12.2	Evaluating Properties	33
12.2.1	Conditionals	33
12.2.2	Evaluation and Inheritance of Properties	33
12.3	Using Components	33
12.4	Using Hooks	33
	Index	35
	General Index	35
	Macro and Environment Index	36
	Property Index	36

1 Quick Start

Thank you for using CoCoTeX.

In this introductory chapter, we are getting familiar with CoCoTeX's design principles and how to use it to define the following boxes:



Info box

This box holds some general information.

Source code:

```
\begin{Info}[\Title{Info box}]
  This box holds some general information.
\end{Info}
```



This box type holds background information if you are interested into the inner workings of the CoCoTeX framework. It can also have a title.

Source code:

```
\begin{Background}
  This box type holds background information if you are interested
  into the inner workings of the {\CoCoTeX} framework. It can also
  have a title.
\end{Background}
```



Alert box

This box holds important information and common error sources.

Source code:

```
\begin{Alert}[\Title{Alert box}]
  This box holds important information and common error sources.
\end{Alert}
```



Alert box with an alternative icon but no title

Source code:

```
\begin{Alert}[\Icon{\faCircleExclamation}]
  Alert box with an alternative icon no title
\end{Alert}
```

As you can see, each of those boxes contain three pieces of information: They may or may not have a **title**, they have an **icon** that can be set freely, and they have a mandatory **content**.

The *general idea* of a “box” is what in the CoCoTeX Framework is called a **Container**. Each box you can see in this pdf is an **Instance** of that Container. Each of those Containers is defined by the three

informational data pieces they share, namely the *Title*, the *Icon* and the *Content*. We call those pieces of Information **Components** of a Container. The instructions, how those Components are typeset is stored in so-called **Properties**. Both Components and Properties are called the **Types** of a Container.

In this documentation, there are three “flavours” of the more abstract Container “Box”, namely *Info*, *Background*, and *Alert*. We call those flavours **Subcontainer** or **Child Containers**, since they **inherit** the Properties and Components from the abstract Box Container.

We start with the most abstract Container **Box**. TO define a new Container, we use the `\ccDeclareContainer` macro which takes two arguments, the *name* of the Container and the *body* of the declaration.

```
\ccDeclareContainer{Box}{%
...
}
```

In the body, we declare the two Types that define this Container using the `\ccDeclareType` macro, which also takes two arguments, the name of the Data Type and its declaration body:

```
\ccDeclareContainer{Box}{%
  \ccDeclareType{Components...}{%
  \ccDeclareType{Properties...}{%
}
```

We start with the Components. Each Component is declared with the `\ccDeclareComponent` macro. This macro has three mandatory argument, although in this tutorial we use only the first one which holds the *name* of the Component.

```
\ccDeclareContainer{Box}{%
  \ccDeclareType{Components}{%
    \ccDeclareComponent{Title}{%}
    \ccDeclareComponent{Icon}{%}
    \ccDeclareComponent{Content}{%}
  }
  \ccDeclareType{Properties...}{%
}
```

As mentioned before, the Properties define how the Components are processed. Let us think about how we could to process our components: Maybe, we want to manipulate the size of the icon and its color. So, let us invent Properties that hold those kinds of information. To define a Property, we use the macro `\ccSetProperty` which takes two arguments, first name of the Property and second its definition. As a convention, Property names are lower-case and words are separated by hyphen:

```
\ccDeclareContainer{Box}{%
  \ccDeclareType{Components...}{%
  \ccDeclareType{Properties}{%
    \ccSetProperty{icon-color}{black}
    \ccSetProperty{icon-size}{\fontsize{24bp}{32bp}}
  }
}
```

Finally, we need to define how the Icons are printed. We want the icons to overlap the box on the left-side and we don’t want the icon to spread the lines. for that, we use the plainTeXprimitives `\llap` and `\smash`. The icon itself is placed in a narrow vbox that grows into its depth (`\vtop`) that we need to lower a bit. Inside that vbox, we print the Icon. Remember that the Icon is a Component. To print the content of a component, we use the `\ccUseComp` macro which take a Component’s name as argument.

```
\ccDeclareContainer{Box}{%
  \ccDeclareType{Components...}{%
  \ccDeclareType{Properties}{%
    \ccUseComp{Icon}
```

```

...
\ccSetProperty{icon-format}{%
  \llap{\smash{\lower1ex\vtop{%
    \hsize32bp
    \ccUseProperty{icon-size}\selectfont%
    \color{\ccUseProperty{icon-color}}%
    \ccUseComp{Icon}%
  }} /vtop
}% /smash
}% /llap
}% /ccSetProperty{icon-format}
}% /ccDeclareType{Properties}
}

```

As you can see, properties are small snippets of $\text{\LaTeX}$ code that sort, format and use Components, but also other Properties.

We also need some properties to process the Title Component. We want the Title to set in bold-face and sans-serif font. For this, we declare a property `title-face`. Another property, `title-format`, combines the `font-face` Property with the Title Component:

```

\ccDeclareContainer{Box}{%
  \ccDeclareType{Components...}{}
  \ccDeclareType{Properties}{%
    \ccSetProperty{title-face}{\bfseries\sffamily}
    \ccSetProperty{title-format}{%
      \bgroup
      \ccUseProperty{title-face}
      \ccUseComp{Title}
    }
  }%
}% /ccDeclareType
}% /ccDeclareContainer

```

In order to make a Container instanciable, we need to declare a corresponding $\text{\LaTeX}$ environment using the `\ccDeclareEnv` macro. This macro takes two arguments, the meaning of which will be explained in a bit.



Strictly speaking, `\ccDeclareEnv` does not define a $\text{\LaTeX}$ environment `<container_name>`, but two macros `\<container_name>` and `\end<container_name>`.

By default, the environment is conveniently called the same as the Container:

```

\ccDeclareContainer{Box}{%
  \ccDeclareType{Components...}{}
  \ccDeclareType{Properties...}{}
  \ccDeclareEnv{}{}
}

```

The environment declaration alone does nothing on its own, yet, but we will look into that later.

Let us first declare out Sub Containers, starting with the Background Container:

```

\ccDeclareContainer{Background}{%
}

```

Now, we need to tell the Background Container that it should inherit the Components and Properties from the Box Container. For that, we use the `\ccInherit` macro, which takes two arguments. First the Types we want to inherit, and second the Container from which we want to inherit the Types.

```
\ccDeclareContainer{Background}{%
  \ccInherit{Components,Properties}{Box}
}
```

Now, we want to specify two things that are prototypical for the Background boxes: They have the Atom symbol as icon, and their color is always green. We can do this by utilizing the Parent Container's Types:

```
\ccDeclareContainer{Background}{%
  \ccInherit{Components,Properties}{Box}
  \ccDeclareType{Components}{\Icon{\faAtom}}
  \ccDeclareType{Properties}{
    \ccSetProperty{icon-color}{green}
  }
}
```

Since we already inherited the Box's Components, we can immediately use the Icon Component and give it a fix value.

By re-declaring an already existing Property name (`icon-color`), we can override the value that came from the Parent Container.

The only thing left to do is to declare Background's environment. For that, we now use the two arguments of the `\ccDeclareEnv` macro. The first argument tells CoCoTeX what should happen at the begin, and the second what should happen before the end of the environment:

```
\ccDeclareContainer{Background}{%
  \ccInherit{Components,Properties}{Box}
  \ccDeclareType{Components...}{}
  \ccDeclareType{Properties...}{}
  \ccDeclareEnv{\begin{Box}}{\end{Box}}
}
```

With that, we define an environment Background that does nothing on its own but to call the parent container's own environment.

We can now start to process the parent's environment. Since `\ccDeclareEnv` already *defines* the Environment macros, we need to use `\RenewDocumentEnvironment` (or `\DeclareDocumentEnvironment`). As mentioned before, we give the environment one mandatory argument for the child container name.

```
\RenewDocumentEnvironment{Box}{ m o +b }{
  \ccEvalType[#1]{Properties}%
  \ccEvalType[#1]{Components}%
  \IfNoValueF{#2}{#2}%
  \Content{#3}%
}{%
  \ccUseProperty{icon-format}%
  \ccIfComp{Title}
    {\begin{mybox}[\ccUseProperty{title-format}]}%
    {\begin{mybox}}
  \ccUseComp{Content}
  \end{mybox}
}
```

2 Overview

Getting started with CoCoTeX

2.1 Installation and Usage

2.1.1 Installation

The current build of the package can be obtained from GitHub:

```
git clone https://github.com/transpect/CoCoTeX.git
```

The actual source files can be found in the `src` sub-folder.

The most recent stable version that is in active use on the *xerif* servers can be found at <https://github.com/transpect/xerif-latex>. Note, however, that is version might be several minor versions behind the source code release and contains additional, xerif-specific, files that are not part of the official CoCoTeX build.

The package is installed via

```
latex cocotex.ins
```

This will create the `cocotex.cls` file, as well as some additional modules that follow the naming convention `coco-<Module>.sty`. These modules will be explained in greater detail below in section 2.1.3 “Usage”.

The documentation of the framework’s source code can be created via

```
lualatex cocotex.dtx
```

Note 1: You *must* use `lualatex` in order to create the source code documentation!

Note 2: The source code documentation is a technical breakdown of the framework’s source code; it is not the same document as the more user-oriented Manual you are currently reading.

2.1.2 Dependencies

CoCoTeX requires a fairly recent LaTeX kernel. It is recommended to use the latest TeXlive build, but not older than texlive 2022, since it uses some newly added concepts like Hooks and Sockets..

The following packages are required by the various CoCoTeX modules:

<code>coco-kernel*</code>	<i>requires</i> <code>etoolbox</code>
<code>coco-common*</code>	<i>requires</i> <code>coco-kernel</code> , <code>xcolor</code> , and <code>graphicx</code>
<code>coco-floats*</code>	<i>requires</i> <code>coco-common</code> and <code>rotating</code> , <code>grffile</code> , <code>footnote</code> , <code>adjustbox</code> , and <code>stfloats</code> and <i>supports</i> <code>tabularx</code> , <code>tabulary</code> , and <code>htmltabs</code> ¹
<code>coco-meta*</code>	<i>requires</i> <code>coco-common</code>
<code>coco-heading*</code>	<i>requires</i> <code>coco-meta</code> , and <code>bookmark</code>
<code>coco-notes*</code>	<i>requires</i> <code>footnote</code> , and <code>endnotes</code>
<code>coco-title*</code>	<i>requires</i> <code>coco-meta</code>
<code>coco-accessibility</code>	<i>requires</i> <code>luaL^AT_EX</code> , <code>coco-kernel</code> , and <code>ltpdfa</code> ² . Older L ^A T _E X kernel versions <i>require</i> <code>atbegshi</code> , <code>xparse</code> , <code>luatexbase-attr</code> , and <code>atveryend</code>

¹The `htmltabs.sty` is included in CoCoTeX’s main GitHub Repository in the `externals/htmltabs/` folder

²`ltpdfa` is included in the `externals/ltpdfa` folder in CoCoTeX’s GitHub repository. Note that CoCoTeX uses only the `.lua` files from that package

<code>coco-lists</code>	<i>requires</i> <code>coco-common</code> , <code>footnote</code> , and <code>endnotes</code>
<code>coco-frame</code>	<i>requires</i> <code>luatex85</code> , and <code>crop</code>
<code>coco-script</code>	<i>requires</i> <code>coco-kernel</code> , <code>babel</code> , <code>fontspec</code> (and therefore <code>luaL^AT_EX</code> or <code>XeL^AT_EX</code>), and <code>filecontents</code>

The CoCoT_EX class file `cocotex.cls` includes most (namely those indicated with an asterisk * in the list above) CoCoT_EX modules and requires *additionally* the `index` and `hyperref` packages. Note that all those packages might have secondary dependencies.

CoCoT_EX itself is designed to run with all L^AT_EX engines, however, in particular the `coco-script` and `coco-accessibility` modules require `luaLATEX`, therefore those modules are either not loaded (`coco-script`), or it is loaded but not activated (`coco-accessibility`) by default.

2.1.3 Usage

CoCoT_EX follows a modular design. It comes with several `.sty` files that can be used independently from another. However, there is also a L^AT_EX Document Class file `cocotex.cls` which can be used to load the whole framework at once.

Using `cocotex.cls`

The `cocotex.cls` serves as stand-in for the L^AT_EX default document classes `article` and `book`. It is called with the usual L^AT_EX command:

```
\documentclass[<options>]{cocotex}
```

The actual document type can be set with the `pubtype` option:

```
\documentclass[pubtype=<mono|article|collection|journal>]{cocotex}
```

The allowed values are:

<code>mono</code>	for monographs, i.e., books that are written by one or multiple authors as a whole,
<code>collection</code>	for books that are collections of contributions of multiple authors, and
<code>article</code>	for single journal articles,
<code>journal</code>	for journals, i.e., collections of multiple journal articles.

Using Single Modules

CoCoT_EX is designed to be used modularly. That means you can use selected modules as packages together with L^AT_EX's default or other third-party document classes. Modules are included like any other package, e.g.,

```
\RequirePackage[<options>]{coco-floats}
\RequirePackage[<options>]{coco-headings}
\RequirePackage[<options>]{coco-title}
```

2.2 Design Goals and Purpose

CoCoT_EX is a programming framework for L^AT_EX developers who need to build and maintain a number of (not too) different publisher-specific style sheets in partly or fully automated typesetting processes. Its original purpose is to serve as a rendering backend for the typesetting tool *xerif*³, but it is also usable as a standalone extension to plain L^AT_EX.

The following features are the main design goals of the CoCoT_EX framework:

³see <https://www.le-tex.de/en/xerif.html>

- Handling of different document types in the same stylesheet:
 - journal articles
 - whole journals
 - chapters by different authors in proceedings and collections,
 - text collections and proceedings, and
 - monographs by (a) single author(s).
- Handling of recurring complex elements that are difficult to set-up using standard- $\LaTeX$, e. g.
 - headings of all levels with authors, subtitles, quotes, etc.;
 - a four-way distinction of material in a heading's title, its pendant in headers and footers, and their entry in the table(s) of contents, and in the PDF bookmarks; and
 - the possibility to provide classes of text components like headings and floats, similar to classes in HTML/CSS; and
 - the structured handling of meta-data, especially for titlepages and accessible PDFs.

To achieve those goals, the framework introduces some concepts into $\LaTeX$ programming that are extensively influenced by object-oriented design principles. The name CoCo $\TeX$ is derived from two of those concepts, namely **C**ontainers and **C**omponents.

The most recent versions of the $\LaTeX$ kernel (Texlive 2024 and later) has seen some quite similar concepts being introduced after they existed before in the form of the `xtemplate` package. The core mechanics of CoCo $\TeX$ and the relationship between CoCo $\TeX$ and $\LaTeX$ Templates are explained in greater detail in 3 “CoCo $\TeX$'s Design Principles”.

2.3 Overview: Modules

As mentioned earlier, CoCo $\TeX$ is modular. The following modules are included in CoCo $\TeX$:

2.3.1 User-Level Modules That Use Containers

<code>coco-headings.sty</code>	The <code>headings</code> module provides a new way to declare and use chapter, section and paragraph titles. It is described in greater detail in 4 “ <code>coco-headings.sty</code> ”.
<code>coco-floats.sty</code>	The <code>floats</code> module provides some extended handling for floating objects like tables or figures. It is described in greater detail in 5 “ <code>coco-floats.sty</code> ”.
<code>coco-title.sty</code>	The <code>title</code> module provides meta data handlers for title pages. It is described in greater detail in 6 “ <code>coco-title.sty</code> ”.
<code>coco-lists.sty</code>	The <code>lists</code> module provides support for list environments. It is described in greater detail in 7 “ <code>coco-lists.sty</code> ”.

2.3.2 User-Level Modules That Do Not Use Containers

<code>coco-frame.sty</code>	provides some helper tools to make the type area visible and add crop marks for printing. It is explained in more detail in ?? “??”.
<code>coco-notes.sty</code>	The <code>notes</code> module handles the easy switching between footnotes and end-notes, as well as the position where and in what way endnotes are printed. It is described in greater detail in ?? “??”.
<code>coco-script.sty</code>	This module provides support for non-latin scripts utilizing Google's Noto fonts. It is described in greater detail in 8 “ <code>coco-scripts.sty</code> ”.
<code>coco-accessibility.sty</code>	The <code>accessibility</code> module provides support to generate PDFs that conform to the PDF/UA standard and interfaces for the <code>ltpdfa</code> package. It will be described in greater detail in 10 “ <code>coco-accessibility.sty</code> ” with some remarks in 3.6 “Accessibility Features”.

2.3.3 Back End Modules

<code>coco-kernel.sty</code>	The <code>kernel</code> module is the heart of the CoCoTeX framework. As such, it is a hard dependency for all other modules and loaded automatically. The kernel module is explained in greater detail in 11 “ <code>coco-kernel.sty</code> ”.
<code>coco-common.sty</code>	The <code>common</code> module is a collection helper macros and functions, that are not per-se part of the CoCoTeX Framework, but utilised by multiple other modules. The common module is loaded automatically by some of the other modules, but not by all. It is explained in greater detail in 12 “ <code>coco-common.sty</code> ”.
<code>coco-meta.sty</code>	The <code>meta</code> module collects methods and concepts that are used by both the <code>title</code> and <code>headings</code> modules. It is therefore auto-loaded by both modules. It is explained in greater detail in 4 “ <code>coco-headings.sty</code> ” and 6 “ <code>coco-title.sty</code> ”, respectively.

3 CoCoTeX's Design Principles

In this chapter, we discuss the major design principles of the CoCoTeX framework, how they are represented in the remainder of the Manual, and introduce some user-level macros to influence the behaviour of those design concepts.

3.1 Containers and Data Types

One design goal of the CoCoTeX Framework is to provide an easy and unified way to configure the typesetting of blocks of inter-connected data.

For instance, take *headings*: They always consist of a *Title*, but also may have some sort of *Numbering*, some have a *Subtitle*, some might have a dedicated *Author*, some are followed by a *Quote* or a *Motto*. They may re-appear (partly) in the head-line of a page, as well as in the table of contents, in some cases with slightly altered data. Another part of the idea behind a heading is that it is always rendered in the same way, for instance, the Title is always bold (or italic or normal) and in a certain font size; if there is an author, it always precedes (or succeeds) the title in this or that font; and so on.

3.1.1 Containers

Container Such a bundle of structured constituents and the instructions how they are presented are called **Containers** in the CoCoTeX framework. In this manual, whenever a pre-defined Container is mentioned, it is colored orange and preceded by the Symbol , for instance Heading. By convention, Container names are (almost) always Capitalized.

Instance One particular manifestation of an abstract Container in a document is called an **Instance** of that Container. For example, the (particular) heading with the (particular) title “Introduction” is an Instance of the abstract concept (or “Container”) “Heading”.

In CoCoTeX, Containers are (usually) LaTeX environments that are (usually) named the same as the Container, thus

```
\begin{Heading}
...
\end{Heading}
```

in the .tex file is an *Instantiation* of the abstract idea (or *Container*) Heading.

The general concept of Containers and Instances is quite similar to the notion of *classes* and *objects* in Object Oriented Programming, whereas *classes* are abstract representations of generalised concepts, and *objects* are concrete instantiations of that abstract class.

3.1.2 Data Types

Type As mentioned above, a Container in CoCoTeX is defined by a specific set of (possible) constituents and a specific set of (conditional) rendering instructions. Those two are fundamentally different in nature, but they share that they define what a Container actually is. CoCoTeX refers to those different sets of Container-defining building blocks as (Data) **Types**. The aforementioned set of structured constituents of a Container form one Type, which we call *Components*, while the set of instructions that tell the engine how those Components are to be processed and rendered form another Type, which we call *Properties*.

3.1.3 Inheritance

Inheritance
Parent Container
Child Container

Containers can be derived from one another by passing all or some Data Types from one Container to another. For instance, the abstract concept “heading” might be extended to various levels, like “section”, “chapter”, “paragraph”, or “part”. Some of the abstract constituents, like `Title` or `Number`, are shared among all those derivations of the concept “heading”, while others might not. E.g. “Author” is usually used only on the “chapter” level, but rarely on “section” or even deeper heading levels. The mechanism to pass certain Types from one Container to another is called **Inheritance**. A Container that is inherited *from* is called the **Parent Container**, the Container that receives the Type from the Parent is called the **Child Container**.

For example, there might be an abstract Container named `Floats` that comprises of the constituents `Caption` and `ListofCaption` for the float’s caption and its entry in the list of figures (or tables), respectively, as well as a bunch of instructions that tell us how the list-of entry is generated from the `Caption`. We can then declare two more Containers, say, `Figure` and `Table`, that both inherit the constituents, as well as the generator instructions from the abstract Container. They then can add their own constituents and instructions, like `Fig` for the image file of the `Figure` Container, or `tabular` for the content body of the `Table` Container. Thanks to the inheritance mechanism, there is no need to define the `Caption` Component and the `ListofCaption` Generator for the two Child containers separately.

In CoCoTeX, Child Containers are usually represented by their own LaTeX environments, e.g.,

```
\begin{Figure}
...
\end{Figure}
```

is an Instance of the Container `Figure`, but Child Containers of the `Heading` Parent are instantiated by an mandatory argument to the parent Container’s environment:

```
\begin{Heading}{chapter}
...
\end{Heading}
```

This is an instance of the Container `chapter`, which in turn is a Child Container of the Parent Container `Heading`. Different modules of the CoCoTeX framework handle instantiations of Containers slightly differently.

3.2 Components

3.2.1 General Concept

Component

As mentioned above, the inter-connected pieces of information that constitute a Container are called **Components** in the CoCoTeX framework. In this manual, Components are marked by the symbol ➡ and colored blue. By convention, Component names are Capitalized or use CamelCase when they are more complex, e.g., ➡ `Author` or ➡ `TocTitle`.

Group Components
Component Group

Some Components can themselves be collections of other Components. An `Author` Component of a `Heading` Container, for instance, can contain a first, a middle and a last name, an academic title, an affiliation, an email address, and many other pieces of information that describe the Author. Those complex Components are called **Group Components**, a Group Component together with its (possible) Child Components is called a **Component Group**.

Counted Component

Some Components may occur multiple times in the same parent Container. A good example are multiple Authors that contributed to the same article in a journal. Those Components are called **Counted Components**. Despite the name, it is not necessarily the case that those Components are enumer-

ated, the name “Counted” merely refers to the way they are processed internally. Due to the way both concepts are implemented in CoCoTeX, Group Components are always also Counted Components.

Collection Component
Overrides

Usually, Counted Components are printed in such a way that all instances of the Component that occur in the same Container Instance are concatenated. The result is stored inside a special Component that is called a **Collection Component**. Those particular Components can be used standalone in spite of the Group Components, and are therefore also called **Overrides**.

As an example, take an Author Group Component, which consists of a `FirstName` and a `LastName` Component. The Collection Component `FullName` is generated from those two Components, but the user could opt to give the `FullName` directly, and therefore *override* what CoCoTeX would otherwise generate from the `FirstName` and `LastName` Components. The same holds for multiple Authors: There is a Collection Component `AuthorNameList` that holds the comma separated list of all the Author’s `FullName` values in the same Container Instance. As with the `FullName` Override earlier, the `AuthorNameList` can as well be given directly, thus eliminating the need to list each author separately.

Note that each Collection Component is always an Override, but not vice-versa: The version of a heading’s title that is printed in the table of contents is usually generated from the `Title` Component of a `Heading (Child)` Container and stored in a Component `TocTitle`. The user can override the `TocTitle` directly by assigning it a value in the `Heading` Instance, therefore, `TocTitle` is an *Override*, but *not* a *Collection Component*, as it is not derived from a Group Component.

Labeled Component

The `coco-meta` module introduces another type of Components, the **Labeled Component**. It is a simple Component with a `Name`, that automatically defines an additional Component `NameLabel`. Usually, Labeled Components are expanded together with the label’s value first, then a separator, and then the simple Component’s value.

3.2.2 How to Use Components

Simple Components

Most Components in Container Instances and Group Components are L^AT_EX macros that take one Argument, e.g.

```
\begin{Figure}
  \Caption{One nice figure.}
  \Fig{\includegraphics{example.eps}}
\end{Figure}
```

In this example, the Container  `Figure` is instantiated by the L^AT_EX environment `Figure` and consists of two Components,  `Caption` and  `Fig`. The Argument of the Component Macros is the Component’s *value*.

\ccComponent

Another way to assign a Value to a Component is to use the `\ccComponent` macro:

```
\begin{Figure}
  \ccComponent{Caption}{One nice figure.}
  \ccComponent{Fig}{\includegraphics{example.eps}}
\end{Figure}
```

It takes two arguments, the first being the name of the Component, the second being the Content that is assigned to the Component in that particular Container Instance.

The `\ccComponent` does check whether a Component with the name in `{#1}` already exists. If no such Component exists, it issues a warning and declares the Component for later use. If the output is unexpected, check the `.log` file or the shell output for CoCoTeX Kernel warnings like that one:

```
[CoCoTeX Kernel Warning] Assigning value to previously undeclared
                          Component '<name>'. Declaring now.
```

This indicates that you tried to assign a value to a Component that wasn't defined earlier, probably because of a typo in the first argument.

Component Groups

In contrast to the simple Components we introduced earlier, Component Groups are usually too complex for simple macros. They are therefore realized as L^AT_EX environments that occur inside the Container Instance environment, for example:

```
\begin{Heading}{Section}
  \ccComponent{Title}{Section Title}
  \begin{Author}
    \FirstName{Jane}
    \LastName{Doe}
  \end{Author}
  \begin{Author}
    \FirstName{John}
    \LastName{Doe}
  \end{Author}
\end{Heading}
```

In this example, the Container  **Section** is instantiated¹ with three Components: one instance of the simple Component  **Title** that is the value *Section Title* assigned to, and two instances of the Counted Component  **Author**. The first Instance has the values *Jane* and *Doe* assigned to the Components  **FirstName** and  **LastName**, respectively, and the second instance has the values *John* and *Doe* assigned to two different instances of the Components  **FirstName** and  **LastName**.  **FirstName** and  **LastName** are both counted components, because they can (and do) occur more than once in the enclosing Container.

When processed, the Values of the Counted Components  **FirstName** and  **LastName** are combined in a special way (more on that later), and the result is stored in a generated Collection Component  **FullName** inside each instance of the Group Component  **Author**. Since Collection Components are also Overrides and Collection Components are not generated when the Override is used explicitly, the following example is equivalent to above's example²:

```
\begin{Heading}{Section}
  \ccComponent{Title}{Section Title}
  \begin{Author}
    \ccComponent{FullName}{Jane Doe}
  \end{Author}
  \begin{Author}
    \ccComponent{FullName}{John Doe}
  \end{Author}
\end{Heading}
```

The two instances of the Group Component  **Author** are further concatenated to a Container-level Collection Component named  **AuthorNameList**. Assuming, the instructions that tell the engine how to build this Collection Component does so by joining the two  **FullName** values with an “and”, the following line is equivalent to both previous examples:

```
\begin{Heading}{Section}
  \ccComponent{Title}{Section Title}
  \ccComponent{AuthorNameList}{Jane Doe and John Doe}
\end{Heading}
```

¹ for the special syntax of  **Heading**'s Child Container Instances, see 3.1.3 “Inheritance” and 4 “coco-headings.sty”

² assuming that the instructions that tell the engine how to build the Collection Component  **FullName** results in the same strings

3.3 Properties

3.3.1 General Concept

Property In the previous section, we mentioned “instructions” that tell the engine how Components are to be processed, and in some instances, combined to other Components. Those instructions are called **Properties**.

While headings of the same level are usually rendered in the same way for a given publisher style, the actual typeface may vary depending on the Components that are actually filled with content for a given heading: A numbered heading might look slightly different than one without a number; a heading with a subtitle might have different spacing than one without a subtitle, and so on.

Properties are mostly short pieces of L^AT_EX code that are usually set by the stylesheet developer according to a publisher’s guidelines and requirements. One of the design goals of CoCoT_EX is to keep the code behind those Properties as comprehensible and pointed as possible. In this manual, pre-defined Properties are indicated by violet text color and the symbol ⚙️, e.g., ⚙️ `title-format`. By convention, Property names are always lower case and words are separated by hyphen.

3.3.2 How to Use Properties

\ccAddToType As mentioned in 3.1.2 “Data Types”, Properties are Types. Most CoCoT_EX modules pre-define Properties. In order to change the behaviour of pre-defined Properties, the Type needs to be appended. This is done with the macro `\ccAddToType`, which takes three arguments:

`{#1}` is the name of the Type
`{#2}` is the name of the Container whose Type list should be altered
`{#3}` is a list of instructions

For instance

```
\ccAddToType{Properties}{Heading}{\foo}
```

add `\foo` to the end of the  `Heading` Container’s Properties list.

\ccAddToProperties For Property lists in particular there exists a shorter macro `\ccAddToProperties` that takes two arguments:

`{#1}` is the name of the Container
`{#2}` is a list of instructions

The line

```
\ccAddToProperties{Heading}{\foo}
```

is equivalent to above’s example.

Defining and Using Properties

In the following, we discuss the macros that can be used in the list of instructions in the last argument of `\ccAddToType{Properties}` and `\ccAddToProperties`, respectively.

\ccSetProperty The most basic way to set the value of a Property is the `\ccSetProperty` macro, that takes two arguments:

`{#1}` the name of the Property to set
`{#2}` the new value of the Property

\ccUseProperty Often, Properties make use of other Properties. In the definition of a Property this is done with the `\ccUseProperty` macro, which takes as its only argument `{#1}`, the name of the Property. In the

following example, this is used to pass the font commands for the main title down to the facility that expands the value of the ➔Title Component:

```
\ccSetProperty{title-face}{\itshape\bfseries}
\ccSetProperty{main-title-format}{%
  \bgroup
    \ccUseProperty{title-face}
    \ccUseComp{Title}
  \egroup}
```

This will internally be resolved to

```
\ccSetProperty{main-title-format}{%
  \bgroup
    \itshape\bfseries
    \ccUseComp{Title}
  \egroup}
```

This mechanism allows Properties to be relatively clear and slender: We can change the font of the ➔Title by changing the ⚙️title-face Property without the need to take care about the expansion and composition of the actual title. And, via Inheritance, we can change the ⚙️title-font once for all Child Containers, while each Child Container might define the ⚙️main-title-format Property slightly differently.

\ccAppToProp To add code to a Property without changing its previous definition, the macros \ccAppToProp and
 \ccPreToProp \ccPreToProp can be used. This allows to add code to the beginning and end of an existing Property, respectively. Both macros take two arguments:

{#1} is the name of the Property
 {#2} is the code that should be added to the beginning or end of the previous definition, respectively.

```
\ccSetProperty{title-face}{\bfseries}
\ccPreToProp{title-face}{\itshape}
\ccAppToProp{title-face}{\large}
```

is equivalent to

```
\ccSetProperty{title-face}{\itshape\bfseries\large}
```

\ccPropertyLet Sometimes, a pre-defined Property's purpose is it to call another Property (for instance, to make the former re-definable without altering the latter). For this, the macro \ccPropertyLet can be used, which takes the names of two Properties as its arguments, and makes Property {#1} an alias of Property {#2}:

```
\ccPropertyLet{toc-number-face}{toc-title-face}
```

in this case, the font of a heading's ➔Number in the table of contents is set to be the same as the font of the entries ➔Title. If we change the ⚙️toc-title-face Property afterwards, ⚙️toc-number-face changes as well, as the line above is equivalent to

```
\ccSetProperty{toc-number-face}{\ccUseProperty{toc-title-face}}
```

\ccPropertyLetX A variant of \ccPropertyLet is \ccPropertyLetX, which fully expands (using \edef) the Property in {#2} before its remainder is assigned to the Property in {#1}. If we change ⚙️toc-title-face after using

```
\ccPropertyLetX{toc-number-face}{toc-title-face}
```

the Property ⚙️toc-number-face retains the original value of ⚙️toc-title-face.

`\ccSetPropertyX` Two more macros to set Properties are `\ccSetPropVal` and `\ccSetPropertyX` which both have the same syntax as `\ccSetProperty` with the difference that `\ccSetPropVal` expands the value once before it is assigned to the property, and `\ccSetPropertyX` expands the value fully before it is assigned to the Property.

Calling Component Values

Returning to our examples from 3.2.2 “How to Use Components”, the instructions that tell the engine how to put together the values of `\ccFirst` and `\ccLast` to generate the value of the `\ccFullName` Component, might look something like that:

```
\ccAddToProperties{Heading}{%
  \ccSetProperty
    {author-full-name-format}
    {\ccUseComp{FirstName}\ccWhenComp{MidName}{ \ccUseComp{MidName}} \ccUseComp{
      LastName}}
}
```

Note that the definition of the Property `\ccauthor-full-name-format` contains a conditional `\ccWhenComp` that takes a Component name as first argument `{#1}` and processes its second argument `{#2}` if and only if the Component has a value assigned to in a given Container instance: If the `\ccMidName` Component is non-empty, it is written in between the `\ccFirst` and `\ccLast` values, which are otherwise put next to each other with a simple space as separator.

`\ccUseComp` The second important macro that is used in this example is `\ccUseComp` which takes as its only argument `{#1}` the name of a Component and is expanded to the value that is assigned to that Component.

`\ccIfComp` A macro similar to `\ccWhenComp` is `\ccIfComp`, which takes three arguments: `{#1}` is the name of the Component, `{#2}` is the branch that is expanded when the Component is assigned a value to in the processed Container, and `{#3}` is the code that is expanded when the Component is *not* assigned a value to within the currently processed Container or Group Component instance.

`\ccUnlessComp` Another variant is `\ccUnlessComp` which also takes two arguments: `{#1}` for the Component’s name, and `{#2}` that is only expanded when the Component is *not* assigned a value in the current instance.

`\ccGetComp` The macro `\ccGetComp` is a combination of `\ccWhenComp` and `\ccUseComp`. It takes one argument `{#1}`, the name of the Component. The two lines in the following example are equivalent:

```
\ccWhenComp{Subtitle}{\ccUseComp{Subtitle}}
\ccGetComp{Subtitle}
```

TODO

Properties consist of two parts, the property’s name and its value. Some Properties provided by the CoCoTeX modules may have a fixed set of string values, while others are completely free to be set and used.

In this manual, the properties provided by the various modules are documented in the following way:

✿ `<name> [<default value>]`

A property with the name `<name>` is set by default to `<default value>`. The user may chose to set it to any of the `<allowed values>`.

`<any>` the user is completely free to set this property to any value she wants.

`<dimen>` It is expected the property to be a dimension. This may be a length or dimension register, a fix value-unit pair that is understood by TeX, or a macro that expands to a dimension/length.

`<num>` It is expected the property to be a numeric value. This may be a counter register, a fix value, or a macro that expands to a number.

<allowed values> without angles mean that those are fixed strings that have a special meaning. Those are explained in the descriptions below the property header.

The “data type” <empty> is used to indicate that the property is un-set or empty. This is the default for some of the properties provided by the CoCoTeX modules, but basically all properties can be set to <empty>.

3.4 Attributes and Style Classes

3.4.1 Attributes

Style Classes Instances of Containers can further be specialized by Attributes and Style Classes. **Style Classes** are comparable to the `class` term that is used in HTML or CSS, respectively. They are a way to further specify Instances of user-level Containers without the need to declare new Child Containers. Container instances of the same Style Class share Properties that diverge somewhat from the standard Property list of their respective Containers.

Attributes In CoCoTeX, **Attributes** are a way to alter the functionality of a Container Instance in a pre-defined way. They are defined for a Container, but called per Instance. Next to Components and Properties, Attributes are the third major Type that defines a Container. Usually, Attributes are predefined key-value pairs or switches without values, and are set by the user in an optional argument of a Container’s LaTeX environment or macro. In this documentation, pre-defined Attributes are colored green and preceded by the symbol , for instance float-pos.

Take, for instance, the Figure Container we discussed earlier. We built in a functionality that allows us to exclude certain Instances from generating a list-of entry, but we need a way to tell LaTeX which instance of the Figure Container should make use of that function. This is where Attributes come in handy: By simply adding a value-less switch nolist to the optional argument of the Figure Container’s environment, we can prevent the list-of entry to be generated. Another example for a valued Attribute would be the float-pos parameter that tells the engine where the float is to be placed (with possible values could be t(op), b(ottom), h(ere), and/or p(age)).

3.4.2 Style Classes

An example for a Style Class would be a pre-defined set of widths that image files are allowed to be printed in. Instead of defining new Figure Containers, each with a Property list that restricts the total width of the printed image either allowed width, we can simply define a style class that overrides one single Property for the one Figure Container. It is noteworthy that Style Classes are usually activated by the class Attribute at the Container Instance.

3.5 Optional Macro and Environment Prefix

If you use CoCoTeX in conjunction with *xerif*, you might notice that in the .tex file that is generated by the converter as an intermediary by-product, Container environments and Components macros all start with the prefix `tp`:

```
\begin{tpFigure}
  \tpCaption{One nice figure.}
  \tpFig{\includegraphics{example.eps}}
\end{tpFigure}
```

This has historic reasons: in an earlier incarnation, when the CoCoTeX framework still had the name *transpect-tex* (“TeX for transpect”), all internal and external macros used that prefix to avoid clashing with macros from other packages and to indicate that the macros and environments were specific to the *transpect* typesetting automation.

However, when the Framework was made standalone and the name changed to CoCoTeX, the `tp` prefix made no longer sense. But in order to keep the output that *xerif* produced without having to change its entire xml-to-tex mapping, we introduced a way to globally attach a prefix to all automatically generated end-user macros and environments: whenever an interface macro is generated (i.e., macros to pass values to Components, and environments that instantiate Containers and/or Component Groups), it is prepended by `\ccPrefix`.

`\ccPrefix`

In order to change the prefix, you can either re-define `\ccPrefix` prior to loading `coco-kernel.sty`, or use the `prefix=<prefix>` package option when loading `coco-kernel.sty`, or the same as class option when loading the `cocotex.cls`.

By default, `\ccPrefix` is defined to be an empty string, but when CoCoTeX is used in conjunction with *xerif*, then the *xerif*-specific file `coco-xerif.sty`³ is loaded, which re-defines the Prefix to `tp`.

For example, the line

```
\documentclass[prefix=my]{cocotex}
```

will cause all generated macros to start with `my`, e.g.,

```
\begin{myFigure}
  \myCaption{One nice figure.}
  \myFig{\includegraphics{example.eps}}
\end{myFigure}
```

However, note that the `\ccComponent` macro takes an *un-prefixed* Component name as its first argument, so the following example is equivalent to the one above

```
\begin{myFigure}
  \ccComponent{Caption}{One nice figure.}           % uses the existing "Caption"
  Component
  \ccComponent{Fig}{\includegraphics{example.eps}} % uses the existing "Fig"
  Component
\end{myFigure}
```

but the next one will probably lead to unexpected behavior:

```
\begin{myFigure}
  \ccComponent{myCaption}{One nice figure.}         % generates a new Component "
  myCaption"
  \ccComponent{myFig}{\includegraphics{example.eps}} % generates a new Component "
  myFig"
\end{myFigure}
```

3.6 Accessibility Features

TODO

³see <https://github.com/transpect/xerif-latex/blob/ally/coco-xerif.sty>

Part I: CoCoT_EX's Container-Based Modules

4 coco-headings.sty

The CoCoT_EX module for headings and table of contents entries

5 coco-floats.sty

The CoCoT_EX module for figures, tables, and other floating objects

6 coco-title.sty

The CoCoT_EX module for titlepages and the document metadata

TODO

7 coco-lists.sty

The CoCoTeX module for numbered, unnumbered, and descriptive lists

Part II:

Additional Modules

8 coco-scripts.sty

The CoCoT_EX module for foreign script support

The `coco-script` module provides the facilities to render non-Standard Average European script systems (henceforth: nSAE).

Those fonts can only be used upright (*not* italic!), and only regular or bold font weights are pre-defined.

The nSAE script systems that are supported by the module so far are listed in Table 8:

Table 1 Script systems supported in the `coco-script` module

Language	Serif Font	Sans Font	Type	usescript parameter	Notes
Arabic	NotoNaskhArabic	NotoSansArabic	ttf	arabic	(see below)
Armenian	NotoSerifArmenian	NotoSansArmenian	ttf	armenian	
Chinese	NotoSerifSC	NotoSansSC	otf	chinese	
Hebrew	NotoSerifHebrew	NotoSansHebrew	ttf	hebrew	
Japanese	NotoSerifJP	NotoSansJP	otf	japanese	

Type *otf* means that the font is included as an Open Type Font; *ttf* means it is a TrueType font

9 coco-endnotes.sty

The CoCoT_EX module for foot- and endnotes

TODO

10 coco-accessibility.sty

CoCoT_EX accessibility features

Part III:

CoCoT_EX's Backend

11 coco-kernel.sty

The CoCoT_EX Kernel module

As we already discussed in chapter 2, Containers are representations of typographical elements that share a more or less fixed set of components that are supposed to be rendered in a similar way.

In this section, we discuss dive deeper into how to declare custom Containers and go through the user-facing CoCoT_EX kernel macros.

`\ccDeclareContainer` A new, custom Container can be declared with the `\ccDeclareContainer` command:

```
\ccDeclareContainer{<name>}{<body>}
```

where `<name>` is the name of the Container, and `<body>` is a list of Container Type Declarations.

`\ccDeclareType` Data Types are declared with the `\ccDeclareType` command:

```
\ccDeclareType{<name>}{<body>}
```

where `<name>` is the name of the Data Type, and `<body>` a list of type-specific variable declarations. The most commonly used Data Types are *Properties*, *Attributes*, and *Components*, but essentially, they can be named anything.

`\ccInherit` Another common command inside the Container body is the `\ccInherit` command. It takes two arguments: `{#1}` is a comma-separated list of Data Types, and `2` is a comma-separated list of Container names. The newly defined Container will then inherit all Data Type declarations from the parent Containers. For instance,

```
\ccDeclareContainer{Parent 1}{%
  \ccDeclareType{Properties}{...}%
  \ccDeclareType{Components}{...}%
  \ccDeclareType{Junk}{...}%
}
\ccDeclareContainer{Parent 2}{%
  \ccDeclareType{Properties}{...}%
  \ccDeclareType{Components}{...}%
}
\ccDeclareContainer{Child}{%
  \ccInherit{Properties,Components}{Parent 1,Parent 2}%
}
```

means that the Container named `Child` will inherit the *Components* and *Properties* Data Types from both Containers `Parent 1` and `Parent 2`, respectively, but not the Data Type `Junk` from `Parent 1`.

Note that both `\ccDeclareType` and `\ccInherit` can only be used inside the body of a Container Declaration!

`\ccIfProp` Sometimes, it is necessary to check whether a Property yields any Content, at all. This can be done with the macro `\ccIfProp`, which takes three arguments:

- `{#1}` the name of the Property
- `{#2}` code that is executed when the Property exists and yields a non-empty string
- `{#3}` code that is executed when either the Property does not exist or if yields an empty string

An example could be:

```
\ccSetProperty{}{}
```

`\ccIfPropVal` Some Properties have pre-defined values. To check for a given value, the macro can be used. It has

four arguments:

- `{#1}` the name of the Property
- `{#2}` the string to compare the Property's expansion against
- `{#3}` code that is executed when the Property's expansion matches `{#2}`
- `{#4}` code that is expanded when either the Property does not exist or its expansion value does not match `{#2}`.

WARNING! Since Properties are LaTeX macros that are always defined `\long`, the comparison value `{#2}` is also stored in a temporary macro using `\long\def`. Therefore, `\ccIfPropVal` cannot be used in contexts that fully expand the Property list, in particular `\ccApplyCollection`!

12 coco-common.sty

Commonly used methods that are not part of the CoCoTeX Kernel

This module of the CoCoTeX Framework is the core that is used by (almost) all other modules in the framework. It provides the basic facilities for Component, Namespace, Hook, and Property handling.

12.1 Using Properties

12.1.1 Setting and Using Property

Properties are set up using one of either commands

```
\ccSetProperty{<name>}{<val>}
\ccSetPropVal{<name>}{<val>}
\ccSetPropertyX{<name>}{<val>}
```

With `\ccSetPropVal`, the second argument is expanded *once* prior to the assignment. Use this if you want to assign the current value of a L^AT_EX control sequence (e.g., a length register) to a property.

`\ccSetPropertyX` is another variant of `\ccSetProperty` except that it expands the value #2 *fully* before assigning it to the property macro with the name #1. Use this if you need to use conditionals to determine the actual values of Properties that otherwise expect fixed named or dimensional values.

When using `\ccSetProperty`, the second argument may not be expanded until the property is evaluated. Use this if <value> is a string.

Values can be retrieved with the command

```
\ccUseProperty{<name>}
```

A user can define her your own properties, but they will most likely have no effect until they are used or evaluated against. Be aware not to wrongly mis-use pre-defined Properties of the CoCoTeX modules.

12.1.2 Default Property Sets

Each Namespace has its own *Default Property Set* that is evaluated before any user-defined specifications or that of child Namespaces are evaluated.

You can extend the default Properties of a <Namespace> by using

```
\ccAddToDefault{<Namespace>}{<setProperties>}
```

where <setProperties> is a list of `\ccSetProperty` assignments.

12.2 Evaluating Properties

12.2.1 Conditionals

There are two conditionals:

```
\ccIfProp{<name>}{<then>}{<else>}
\ccIfPropVal{<name>}{<value>}{<then>}{<else>}
```

The first conditional `\ccIfProp` checks if a property with the name `<name>` is defined *or* set to a value other than an empty pair or brackets. If the macro is defined and non-empty, the instructions in the `<then>` branch are executed. Otherwise, the instructions in the `<else>` branch are executed.

The conditional `\ccIfPropVal` also checks if the Property is defined and also if the current Value of that Property is equal to `<value>`. Both `<value>` and the Value of the Property are fully expanded before the comparison. If both expansions are equal, `<then>` is executed, otherwise `<else>` is executed.

12.2.2 Evaluation and Inheritance of Properties

The evaluation of macros is done with the macro:

```
\ccCascadeProps{<namespace>}{<top-level-namespace>}
```

This macro recursively loads a `<namespace>`'s properties, the properties of the namespace's parent, and the default properties of the `<top-level-namespace>` (in reverse order).

12.3 Using Components

The macro

```
\ccUseComp{<Component>}
```

is used to expand the contents of a Component with the name `<Component>`.

The conditional

```
\ccIfComp{<Component>}{<True>}{<False>}
```

can be used to check the value of a Component with the name `<Component>`.

If the Component's content expands to `\relax`, `<False>` is expanded. This is automatically the case if a Component is *not used* in a Namespace environment, or if a Component with the name `<Component>` is undefined.

In all other cases, `<True>` is expanded. It is also the case, when the argument of the Component macro is left empty.

12.4 Using Hooks

Hooks are Namespace-specific, yet globally defined, landing points for custom adjustments.

In contrast to Properties, which are replaced when re-declared, hooks are cumulated, i.e., if you add material into a hook, its previous content will remain.

The second big difference is the scope: Hooks are global, that is they can be filled at every point in the tex code. Properties, on the other hand, can only be altered within certain macros or environments.

A Hook is declared with the `\ccDeclareHook` macro which takes the Hook's name as argument:

```
\ccDeclareHook{<Hook-Name>}
```

To add code to a hook, simply use

```
\ccAddToHook[<namespace>]{<Hook-Name>}{<Appendix>}
```

where `<Hook-Name>` is the name of the Hook, and `<Appendix>` is the code you'd like to add to the hook. If the optional `<namespace>` argument is omitted, the hook will be bound to the currently active Namespace.

The Hook is used with the macro

```
\ccUseHook{<Hook-name>}
```

Make sure that `\cc@namespace` is set correctly or else you may expand the wrong Hook.

A Hook can be re-used.

Index

In general, page entries in **bold face** refer to main sections that describe the index term in greater detail. Roman page numbers refer to pages where the entry is used.

General Index

A		L	
Accessibility	6	Labeled Component	10
Attributes	15	Lists	6
C		M	
\ccIfProp	27	Meta	7
\ccIfPropVal	27	Module	
Child Container	9	Accessibility	6
Class Options		Common	7
prefix	16	Floats	6
coco-common	29	Frame	6
coco-endnotes	24	Headings	6
coco-floats	19	Kernel	7
coco-headings	18	Lists	6
coco-scripts	23	Meta	7
coco-title	20	Notes	6
Collection Component	10	Title	6
Common	7	N	
Component	9	NamespaceChild	29
Component Group	9	Notes	6
ComponentUsage	30	O	
Container	8, 27	Overrides	10
Counted Component	9	P	
D		Parent Container	9
Default Property Set	29	prefix	<i>see</i> Class Options
Dependencies	4	PropertiesConditionals	30
F		PropertiesDefault	29
Frame	6	PropertiesEvaluation	30
G		PropertiesInheritance	30
Group Components	9	PropertiesSetup	29
H		PropertiesUsage	29
Headings	6	Property	12
HooksUsage	30	pubtype	<i>see</i> Class Options
I		S	
Inheritance	9	Style Classes	15
Installation	4	T	
Instance	8	Title	6
K		Type	8
Kernel	7	U	
		Usage	5

Macro and Environment Index

In this index, the `cc(®)`-Prefixes are omitted when sorting the entries.

A		P	
<code>\ccAddToProperties</code>	12	<code>\ccPrefix</code>	16, 16, 16
<code>\ccAddToType</code>	12	<code>\ccPreToProp</code>	13
<code>\ccApplyCollection</code>	28	<code>\ccPropertyLet</code>	13, 13
<code>\ccAppToProp</code>	13	<code>\ccPropertyLetX</code>	13
C		S	
<code>\ccComponent</code>	10, 10, 16	<code>\ccSetProperty</code>	12, 14
D		<code>\ccSetPropertyX</code>	14, 14
<code>\ccDeclareContainer</code>	27	<code>\ccSetPropVal</code>	14, 14
<code>\ccDeclareType</code>	27, 27	U	
G		<code>\ccUnlessComp</code>	14
<code>\ccGetComp</code>	14	<code>\ccUseComp</code>	14, 14
		<code>\ccUseProperty</code>	12
I		W	
<code>\ccIfComp</code>	14	<code>\ccWhenComp</code>	14, 14, 14
<code>\ccInherit</code>	27, 27		

Property Index

In this index, the specification in parentheses denote the (Parent) Container inside of which the Property is valid.

F		S	
<code>float-number ()</code>	22	<code>sub-number ()</code>	22